

PHVE: a bijective, prefix-searchable spatial code for anatomical volumes

Paul Guindo

Altius Academy SNC, Échallens, Vaud, Switzerland

paulguindo@altius-group.ch

August 5, 2026

Abstract

We describe PHVE, a spatial code that assigns to every point of a standardised anatomical reference volume a short string over a 29-character alphabet. The code is bijective on the voxel grid at sufficient order, and it is *hierarchical*: truncating a code to its first k characters yields the code of the enclosing dyadic cell, so a string prefix denotes exactly a region of space. The construction is the Hilbert index of Skilling composed with an affine normalisation and a radix-29 expansion; no novelty is claimed for any of these ingredients.

What the software contributes is the packaging and its verification. We report an exhaustive collision test on the MNI152 brain mask (235 375 voxels), which fixes the order at which the code becomes injective, and a benchmark of dyadic-cell range queries showing that a plain ordered index over an integer column answers them with two binary searches, independently of the number of results, against a k -d tree that must assemble a scattered set. Every figure in this paper is produced by a seeded script in the accompanying repository.

We are equally explicit about what the code does *not* do. A companion experimental campaign refuted several claims made for this construction in earlier drafts — bandwidth reduction, a compression gain, and an inter-patient stability property — and Section 4 states each refutation with a pointer to the script that produced it. The code is offered as an indexing and addressing tool, and as nothing more.

Keywords: space-filling curves, Hilbert index, spatial indexing, medical image addressing, prefix search, reproducible software.

1 Introduction

Addressing a location inside a body is awkward. Millimetre coordinates in a template space are precise but unreadable, carry no notion of scale, and cannot be compared as strings. Anatomical names are readable but coarse and ambiguous. Databases that must answer “what is stored near here” usually reach for a spatial index — an R-tree or a k -d tree — which means an extension or an auxiliary structure that a plain relational column does not provide.

Geographic information systems solved the analogous problem long ago with hierarchical short codes: Geohash, S2, H3, what3words. A location becomes a string, a shorter string is a coarser region, and containment is a prefix test, so an ordinary B-tree suffices. PHVE (*Parametric Hilbert Volume Encoding*) applies the same recipe to anatomical reference volumes.

What is and is not new. The Hilbert curve, its Skilling construction [1], and radix encodings of space-filling-curve indices are classical, and hierarchical geographic codes are widely deployed. Nothing in Section 2 is new mathematics. The contribution of this paper is a specific, verified, reproducible instantiation for anatomy: a reference family of volumes, an alphabet chosen to avoid transcription errors, an exhaustive determination of the order at which the code is

injective on a standard brain mask, and a measured statement of what prefix queries cost. We are explicit about this because an earlier draft of this work claimed more, and much of it did not survive testing (Section 4).

2 Construction

2.1 Reference family

A *reference volume* is an axis-aligned box $V_\alpha = \prod_{i=1}^3 [a_i, b_i] \subset \mathbb{R}^3$ with an identifier α , fixed once and published. Table 1 lists the family used here. Coordinates are millimetres in the template space of the volume; for the cranial volume **CR** this is MNI space.

Code	Volume	Dimensions (mm)
CR	Cranium	$300 \times 250 \times 250$
TX	Thorax	$400 \times 350 \times 300$
CA	Heart	$150 \times 120 \times 100$
AB	Abdomen	$400 \times 350 \times 300$
MB	Limb	$700 \times 200 \times 200$
FB	Full body	$2000 \times 600 \times 400$

Table 1: The reference family. The identifier is part of the code, so a code is meaningless without it and cannot be misread across volumes.

2.2 The map

Fix an order $p \in \mathbb{N}$ and write $n = 2^p$. The encoding is the composition

$$F_{\alpha,p} = \text{Enc} \circ \text{Hil}_p \circ \text{Nor}_{\alpha,p},$$

of three maps.

Normalisation. $\text{Nor}_{\alpha,p} : V_\alpha \rightarrow \{0, \dots, n-1\}^3$ sends $\mathbf{x}$ to the integer cell

$$g_i = \min\left(n-1, \left\lfloor \frac{x_i - a_i}{b_i - a_i} n \right\rfloor\right), \quad i = 1, 2, 3.$$

The cell edge lengths are $\Delta_i = (b_i - a_i)/n$, which differ between axes when the box is not cubic.

Hilbert index. $\text{Hil}_p : \{0, \dots, n-1\}^3 \rightarrow \{0, \dots, 2^{3p} - 1\}$ is the Skilling index [1], a bijection between the grid and an interval of integers.

Radix expansion. Enc writes the integer in base 29 over the alphabet

$$\Sigma = 23456789\text{ABCDEFGHIJKLMNPQRSTUVWXYZ},$$

zero-padded to $k(p) = \lceil 3p \log 2 / \log 29 \rceil$ characters. The alphabet omits 0, 1, I, L, O, U and the lower case, so that no two characters are confusable when a code is read aloud, handwritten, or scanned. At $p = 8$ a code is 5 characters, and a full address reads

$$\text{CR:GVP-5X}$$

for the point (0, 50, 30) mm in MNI space, the hyphen being cosmetic.

2.3 The three properties that matter

Proposition 1 (Bijectivity). *For every p , Hil_p is a bijection, hence $F_{\alpha,p}$ is injective on the set of grid cells and a code determines its cell exactly.*

This is a property of the Skilling construction, not of ours. What is not automatic, and is the practical question, is whether distinct *voxels* of a given image fall in distinct cells; this is settled empirically in Section 3.2.

Proposition 2 (Truncation is containment). *Let $c = F_{\alpha,p}(\mathbf{x})$ and let $c|_k$ denote its first k characters. Then the set of points whose code shares the prefix $c|_k$ is exactly a dyadic sub-box of V_α containing $\mathbf{x}$, and these sub-boxes are nested as k decreases.*

Proof sketch. The Hilbert index has the property that the $2^{3\ell}$ cells of a dyadic sub-box of level ℓ occupy a contiguous, aligned block of indices of length $2^{3(p-\ell)}$. A prefix in base 29 fixes the leading digits, hence an aligned block of integers; the alignment of the two block structures is what makes the correspondence exact at the levels where 29^j and $2^{3\ell}$ align, and a nesting elsewhere. $\square$

Proposition 2 is the reason the code is useful: *a region query is a string prefix*, so it is a range scan on an ordinary index.

Proposition 3 (Quantisation). *Decoding a code to the centre of its cell yields a point within $\frac{1}{2}\|\Delta\|_2$ of the original, and this bound is attained.*

The bound is elementary; that it is *attained* was checked numerically, the measured maximum error agreeing with the theoretical value to all printed digits at $p \in \{6, 8, 10, 12\}$.

Remark 4 (Which Hilbert curve). There are 10 694 807 structurally distinct three-dimensional Hilbert curves [3]. This implementation uses Skilling’s. In $d = 2$ it coincides exactly with the index of Hamilton and Rau-Chaplin [2]; in $d = 3$ it does *not* — the two induce different total orders already at $p = 1$, and no composition with any of the 48 signed axis permutations of the cube (with or without reversal of the traversal) reconciles them for $p \geq 2$. Any locality constant quoted for this software is therefore a property of the Skilling variant and must not be compared with published values for other variants. In particular, the worst-case L_2 dilation is 6 in $d = 2$ [4] but approximately 29.5 for the Skilling variant in $d = 3$, against 22.9 for the best published three-dimensional curve [3]: the widely implemented variant is about 24% worse. This matters for users, because that constant is what governs how tightly a prefix confines a point.

3 Verification

All scripts are in `experiments/` of the accompanying repository, seeded, and write a JSON record of every number they produce. The machine used is an AMD Ryzen 7 7840HS (8 cores, L1d 32 KiB/core, L2 1 MiB/core, L3 16 MiB shared), Linux 6.17, CPython 3.12.3, NumPy 2.4.2, SciPy 1.17.1.

3.1 Kernel

The vectorised Skilling kernel is bijective with an exact round trip and unit-step adjacency for $d \in \{2, 3\}$ and $p \leq 5$, and agrees with the repository’s independent scalar implementation on all 4096 points at $d = 3$, $p = 4$.

3.2 Injectivity on a real volume

The practical question is at which order the code separates the voxels of an actual image. We encoded the centre of every voxel of the MNI152 brain mask at 2 mm (`nilearn`, 235 375 voxels, all inside the CR box) and counted distinct codes.

p	cell (mm)	colliding voxels	rate	max multiplicity
6	$4.69 \times 3.91 \times 3.91$	207 171	88.02%	12
7	$2.34 \times 1.95 \times 1.95$	33 183	14.10%	2
8	$1.17 \times 0.98 \times 0.98$	0	0.00%	1
9, 10	—	0	0.00%	1

Table 2: Collisions on the MNI152 brain mask. Injectivity begins exactly at $p = 8$, which is where the cell first fits inside the 2 mm voxel along every axis. Script: `exp15_v4_carryover.py`.

The threshold is not a surprise — it is where $\Delta_i < 2$ mm for all i — but it had never been measured, and $p = 8$ is therefore the smallest order at which a code may be used as a voxel identifier for this template. At $p = 7$ one voxel in seven shares its code with a neighbour.

3.3 Prefix range queries

By Proposition 2 the points inside a dyadic cell occupy a contiguous range of indices. On a sorted integer column the query is two binary searches and a slice; the client supplies the prefix, so no Hilbert computation happens at query time. We compare against a `scipy` k -d tree on the same points, using a Chebyshev ball, which *is* the axis-aligned box, so both methods answer the same question exactly.

We stress that this is a *restricted* query class. A k -d tree answers arbitrary boxes and balls; the prefix index answers dyadic cells only. Comparing them on arbitrary boxes would be meaningless. The claim under test is narrow: for the queries the encoding is designed for, no spatial index is needed.

Before any timing was recorded, both methods were checked to return identical point sets; over the 540 queries reported, they always did.

depth	cell (voxels/side)	median results	prefix (ms)	k -d tree (ms)	ratio
2	64	6849	0.0048	0.4232	87.3
3	32	1580	0.0034	0.1239	36.3
4	16	478	0.0030	0.0572	19.0
5	8	64	0.0028	0.0206	7.5

Table 3: Dyadic-cell queries on the 235 375-voxel brain mask at $p = 8$; medians over 24–200 queries per depth, all on non-empty cells. Script: `exp22_prefix_index.py`.

The structural point, rather than the ratio, is what should be taken away. The prefix query cost is essentially independent of how many points it returns (0.0028–0.0048 ms across a hundredfold change in selectivity), because the answer is a contiguous *range* rather than a set: locating it costs $O(\log N)$ and reading it is a sequential scan. The k -d tree must assemble scattered indices, so its cost grows with the output.

Where the prefix index loses. It is *not* cheaper to build: 0.123 s against 0.056 s for the k -d tree, since encoding and sorting is more work than a k -d tree construction. It is smaller (3.6 MiB against at least 5.4 MiB for the coordinate array alone), and it requires no spatial

extension — an integer or string column in any database will do — but the build cost is a genuine disadvantage and we report it as such.

4 What this code does not do

An earlier draft of this work advanced several further claims. A systematic experimental campaign refuted them. We list them here, with the script that settled each, because a software paper that omits its own negative results is not usable by anyone.

- **It does not reduce matrix bandwidth.** Renumbering the vertices of a finite-element mesh by this code leaves the maximum index gap at $\Theta(N)$, essentially unchanged from the natural ordering, and worse than reverse Cuthill–McKee by more than an order of magnitude. What it reduces is the *mean* and *median* gap. Moreover the renumbering is analytically inert: it is a permutation, so the spectrum, the conditioning and the solution are unchanged, verified to 10^{-12} .
- **It does not accelerate a sparse solver in any general sense.** With a permutation-equivariant preconditioner the iteration count is invariant by construction (exactly so for a Chebyshev polynomial, over all sizes and seeds tested). The only measured advantages are a 35–43% reduction in *simulated* L1 cache misses above $N \approx 3 \cdot 10^4$, with no effect at L2 or L3, and an ordering that is 1.5–4.3× cheaper to compute than reverse Cuthill–McKee. With a fixed-pattern incomplete Cholesky, which ordering gives fewer iterations was found to depend on the mesh seed and could not be determined.
- **It does not improve DPCM compression** of the volumes tested. On MNI152 the Hilbert traversal loses to a raster scan. An earlier reported gain came from a script that quantised a signal on $[0, 1]$ with `round`, reducing it to two levels before taking residuals.
- **It is not the compact Hilbert index** of Hamilton and Rau-Chaplin in three dimensions (Remark 4), although in two dimensions it is identical to theirs.
- **Its stability under coordinate jitter is quantisation, not anatomy.** An experiment reported as “inter-patient stability” uses no inter-patient data: it perturbs fixed landmarks with Gaussian noise and counts unchanged codes. Over $p \in \{5, \dots, 8\}$ and $\sigma \in \{0.25, \dots, 4\}$ mm the measured rate agrees with the elementary prediction that a point merely has to remain inside its own cell to within 0.05, and to within 0.01 for $\sigma \geq 2$ mm. It is a property of the grid. Whether codes agree across registered subjects is an open question that this software does not answer.

5 Software

Availability. The reference implementation, the demonstration application, and every script cited above are available at <https://github.com/Altius-Academy-SNC/PHVE> under the MIT licence.

Interface. The core is three functions.

```
from codec3d import encode, decode, truncate

code = encode(x_mm, y_mm, z_mm, p=8, volume="CR")    # -> 'CR:GVP-5X'
xyz, resolution_mm = decode(code)                    # cell centre + scale
region = truncate(code, n_chars=3)                   # enclosing dyadic cell
```

Use in a database. Because a region is a prefix, no spatial extension is required:

```
CREATE INDEX ON findings (phve_code);  
SELECT * FROM findings WHERE phve_code LIKE 'CR:GVP%';
```

Demonstration. A Streamlit application renders the traversal on a brain volume, the effect of truncation on the located region, and the prefix query of Section 3.3.

Reproducibility. Every number in this paper comes from a seeded script that writes a JSON record to `experiments/results/`. The laboratory notebook `experiments/LOGBOOK.md` records the runs in chronological order, including the failures and the two bugs found in earlier scripts; `experiments/UNVERIFIED.md` lists what remains untested.

6 Limitations

The code addresses a *reference* volume. Using it for a subject requires registering that subject to the template, and the registration error is then added to the quantisation error of Proposition 3; this paper measures neither. The family of Table 1 is fixed by convention and its boxes are not derived from any anatomical criterion. Injectivity was established for one template at one resolution (Table 2); other templates will have their own threshold, computable by the same script. Finally, the benchmark of Section 3.3 is single-threaded, in-memory, and on one dataset; it says nothing about behaviour on disk or under concurrency.

References

- [1] J. Skilling. Programming the Hilbert curve. *AIP Conference Proceedings* **707** (2004), 381–387.
- [2] C. H. Hamilton and A. Rau-Chaplin. Compact Hilbert indices: space-filling curves for domains with unequal side lengths. *Information Processing Letters* **105** (2008), 155–163.
- [3] H. Haverkort. An inventory of three-dimensional Hilbert space-filling curves. arXiv:1109.2323 (2011); and How many three-dimensional Hilbert curves are there? arXiv:1610.00155 (2016).
- [4] B. Moon, H. V. Jagadish, C. Faloutsos and J. H. Saltz. Analysis of the clustering properties of the Hilbert space-filling curve. *IEEE Transactions on Knowledge and Data Engineering* **13** (2001), 124–141.
- [5] C. Gotsman and M. Lindenbaum. On the metric properties of discrete space-filling curves. *IEEE Transactions on Image Processing* **5** (1996), 794–797.